SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192519100**
Name	**Niranjna devi S I**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I Niranjna devi S I /192519100 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Niranjna devi S I

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

ANSWERS

1. Hospital Doctor–Shift Assignment Using Backtracking Search:

Problem: Assign three doctors (D1, D2, D3) to Morning, Afternoon, and Night shifts while satisfying all the given constraints.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3.
3. Only one doctor is allowed per shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Step-by-step backtracking

Step	Assignment	Constraint check	Result
1	D1 → Morning	D1 is not assigned Night.	Valid
2	D2 → Afternoon	D2 is before D3, if D3 is assigned Night.	Valid
3	D3 → Night	D3 is not assigned Morning.	Valid
4	Check all shifts	Each shift has one doctor.	Valid
5	Check all doctors	All doctors have exactly one shift.	Valid

```
BACKTRACK(assignment):

   IF all doctors are assigned:
      IF all constraints are satisfied:
         RETURN assignment
      ELSE:
         RETURN FAILURE

   Select an unassigned doctor D

   FOR each available shift:
      Assign D to the shift

      IF no constraint is violated:
         result = BACKTRACK(assignment)

         IF result != FAILURE:
            RETURN result
```

Remove the assignment # Backtrack

RETURN FAILURE

Final valid schedule:

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Conclusion: Backtracking finds a valid schedule by assigning doctors one at a time and checking constraints at each step.

2. Robot Grid Navigation Using BFS Search:

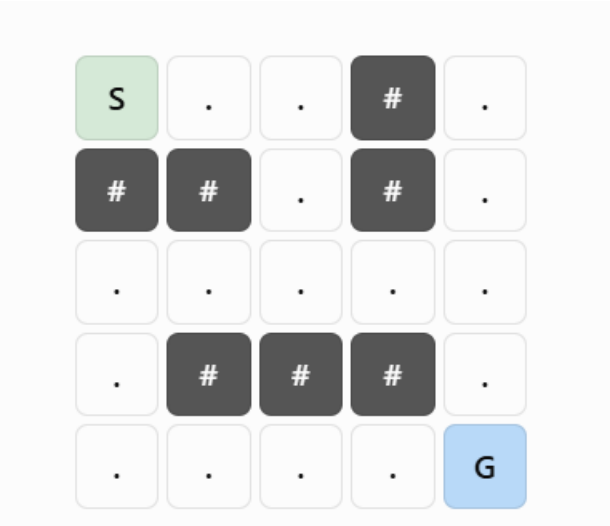
Problem: A robot must move from Start (S) to Goal (G) in a 5 × 5 grid without crossing obstacles.

Given constraints:

- Allowed movements: Up, Down, Left, Right.
- Cost per move = 1.
- Obstacles cannot be crossed.
- Manhattan distance is used as a heuristic.

Example grid

- S = Start
- G = Goal
- **= Obstacle**
- . = Free cell



Manhattan distance heuristic

BFS explores nodes level by level, using a FIFO queue. Manhattan distance is not used to prioritize nodes in ordinary BFS; it is commonly used with A* search.

```
BFS(grid, start, goal):

    queue = [start]
    visited = {start}
    parent[start] = None

    WHILE queue is not empty:
        current = queue.pop_front()

        IF current == goal:
            RETURN reconstruct_path(parent, goal)

        FOR each neighbor in Up, Down, Left, Right:
            IF neighbor is inside grid
               AND neighbor is not an obstacle
               AND neighbor not in visited:

                visited.add(neighbor)
                parent[neighbor] = current
                queue.push_back(neighbor)

    RETURN "No path exists"
```

Step-by-step node expansion:
For the example grid, using the order Right, Down, Left, Up to break ties:

Step	Expanded node	Newly discovered nodes
1	S (1,1)	(1,2)
2	(1,2)	(1,3)
3	(1,3)	(2,3)
4	(2,3)	(3,3)
5	(3,3)	(3,4)
6	(3,4)	(3,5)
7	(3,5)	(2,5), (4,5)
8	(2,5)	(1,5)
9	(4,5)	(5,5)
10	(1,5)	—
11	(5,5)	Goal reached

Optimal path:

$$S \to (1,2) \to (1,3) \to (2,3) \to (3,3) \to (3,4) \to (3,5) \to (4,5) \to G$$

3. Autonomous Rescue Robot – Online Search:

a) Analysis of constraints

The rescue robot operates in a disaster-affected building and must reach a survivor while avoiding obstacles and minimizing risk.

Constraint	Effect on decision-making
Movement in four directions	Limits the available actions to Up, Down, Left, and Right.
Obstacles	The robot must avoid blocked cells.
Limited sensing	The robot only knows the status of adjacent cells and cannot initially observe the entire building.
Movement cost = 1	Every normal movement adds one unit to the path cost.
Risky-zone cost = 2 extra	Entering a risky cell costs 3 units in total.
Dynamic knowledge	The robot updates its map whenever it receives new information.
Safety requirement	The robot should avoid dangerous routes when safer alternatives exist.

The total path cost is:

TOTAL COST = NUMBER OF MOVES + 2(RISKY CELLS ENTERED)

b) Solution using Uniform Cost Search and an Online Search Agent:

Chosen approach: An online search agent that updates its map as it explores, using Uniform Cost Search (UCS) on the known portion of the environment.

Steps involved

1. Initialize: Set the robot's current position to S and initialize its known map.
2. Sense: Observe adjacent cells to identify obstacles and risky zones.
3. Update the map: Record newly discovered information.
4. Calculate costs: Assign cost 1 to normal movement and cost 3 to movement into a risky cell.
5. Search: Use UCS to select the known route with the lowest cumulative cost.
6. Move: Execute the first action of the selected route.
7. Repeat: Sense the surroundings, update the map, and replan if necessary.
8. Terminate: Stop when the robot reaches G and the survivor is located.

```
ONLINE_RESCUE_AGENT(S, G):

    position = S
    known_map = initialize_map()
    total_cost = 0

    WHILE position != G:

        observations = SENSE_ADJACENT_CELLS(position)
        UPDATE_MAP(known_map, observations)
```

```
    path = UNIFORM_COST_SEARCH(
            known_map, position, G,
            normal_cost=1,
            risky_extra_cost=2
          )

    IF path does not exist:
       DISPLAY "No known safe route"
       WAIT or EXPLORE_SAFE_NEIGHBOR()

    ELSE:
       next_cell = path[1]

       IF next_cell is blocked:
          CONTINUE

       IF next_cell is risky:
          total_cost = total_cost + 3
       ELSE:
          total_cost = total_cost + 1

       MOVE_TO(next_cell)
       position = next_cell

  DISPLAY "Survivor reached"
  DISPLAY total_cost
```

c) Application of AI concepts:

1. Intelligent agent:
The robot perceives its surroundings through sensors, processes information using its internal map, and performs actions to reach the survivor.

- Sensors: Detect obstacles and risky zones.
- Agent program: Updates the map and selects routes.
- Actuators: Move the robot in the chosen direction.

2. Rationality:
A rational agent selects actions that maximize expected performance based on the information currently available.
The robot should choose routes that minimize movement costs and risks while maintaining safety. Since its observations are incomplete, it must update its decisions when new information becomes available.

3. Environment type:

Property	**Classification**
Observability	Partially observable
Determinism	Stochastic or uncertain
Episodic or sequential	Sequential
Static or dynamic	Dynamic if hazards or obstacles can change
Discrete or continuous	Primarily discrete grid navigation
Number of agents	Single-agent, assuming no other robots

d) Justification of the chosen approach:
An online search agent combined with UCS is suitable because the robot cannot observe the complete environment in advance. It must gather information, update its map, and replan as it moves.
UCS considers the cumulative cost of every route, including additional costs for risky cells. It therefore finds the minimum-cost path within the currently known map when costs are nonnegative and the search is allowed to complete.
However, UCS alone cannot guarantee global safety in an unknown environment. The robot should use safety checks, avoid unverified dangerous areas when possible, and replan when new obstacles or hazards are discovered.

Conclusion: Combining online search, UCS, and continuous sensing allows the rescue robot to navigate an uncertain environment while balancing path cost and safety.